DOCE ENCANTO

E-commerce acadêmico de cupcakes
Documentação de projeto e implementação

Matheus da Silva Mattos
RGM 36338770

São Paulo • 2026

Resumo executivo

O Doce Encanto é uma aplicação web responsiva para venda demonstrativa de cupcakes. Reúne cadastro, autenticação, catálogo, carrinho em sessão, checkout com pagamento simulado, histórico e administração. Foi desenvolvida em Django 5.2 LTS, com SQLite local e configuração PostgreSQL/Render.

Este relatório consolida análise, requisitos, backlog, UML, banco de dados, interface, testes, laudo, implantação, manuais e roteiros. O projeto é acadêmico e não processa pagamentos reais.

1. Visão, escopo e metodologia

Objetivo: demonstrar um fluxo de compra claro e seguro para uma confeitaria artesanal. Inclui contas, catálogo, carrinho, pedidos e administração; exclui gateway real, cupons, avaliações públicas, frete por distância, estoque complexo e aplicativo nativo.

Etapa	Atividade	h
Fundação	Projeto Django, configurações, modelos e migrações	8
Contas	Cadastro, login, logout e permissões	5
Catálogo	Listagem, destaques, imagens e administração	6
Carrinho	Sessão, adicionar, atualizar, remover e totais	8
Pedidos	Checkout transacional, histórico e detalhes	10
Qualidade	Testes, cobertura, smoke responsivo e correções	9
Entrega	README, UML, manual, Render e roteiros	10



Figura 1 — Mapa de afinidade.

2. Histórias de usuário

Prioridades e pontos orientaram a implementação. Regras transversais: preço recalculado no servidor, 1–20 unidades, produtos ativos, isolamento por usuário, transação atômica e pagamento demonstrativo.

ID	História	Formulação e aceitação	P	Pts
US01	Criar conta	Como visitante, quero criar uma conta com usuário e senha, para acompanhar meus pedidos. Dados obrigatórios válidos; usuário não pode se repetir; senha confirmada.	Alta	3
US02	Entrar no sistema	Como cliente, quero autenticar-me, para acessar checkout e histórico. Credenciais válidas redirecionam; inválidas exibem erro sem revelar qual campo falhou.	Alta	2

ID	História	Formulação e aceitação	P	Pts
US03	Sair do sistema	Como cliente, quero encerrar minha sessão, para proteger minha conta. Logout remove autenticação e mantém navegação pública disponível.	Alta	1
US04	Ver catálogo	Como visitante, quero consultar cupcakes ativos, para escolher o que comprar. Somente ativos aparecem; nome, descrição, imagem e preço são exibidos.	Alta	3
US05	Adicionar ao carrinho	Como visitante, quero adicionar um cupcake, para montar meu pedido. Produto ativo é incluído; quantidade final fica entre 1 e 20.	Alta	3
US06	Alterar quantidade	Como visitante, quero ajustar a quantidade de um item, para corrigir meu pedido. Quantidade aceita de 1 a 20; total é recalculado no servidor.	Alta	2
US07	Remover item	Como visitante, quero remover um item do carrinho, para desistir de um produto. Item deixa a sessão e totais são atualizados.	Alta	1
US08	Revisar carrinho	Como visitante, quero ver itens, quantidades e total, para confirmar a compra. Carrinho vazio tem estado orientativo; preços vêm do banco.	Alta	3
US09	Finalizar pedido	Como cliente, quero informar endereço e pagamento simulado, para registrar meu pedido. Checkout autenticado, validado e transacional cria pedido e itens completos.	Alta	8
US10	Informar observações	Como cliente, quero registrar instruções de entrega, para personalizar orientações. Campo opcional aceita texto limitado e é preservado no pedido.	Média	1
US11	Ver meus pedidos	Como cliente, quero consultar meu histórico, para acompanhar compras. Lista contém apenas pedidos do usuário autenticado.	Alta	3
US12	Ver detalhe do pedido	Como cliente, quero abrir um pedido meu, para conferir itens e status. Acesso a pedido alheio retorna 404; snapshot de preços é exibido.	Alta	3
US13	Gerenciar produtos	Como administrador, quero cadastrar, editar e inativar cupcakes, para manter o catálogo. Admin permite CRUD; produto inativo não é vendido.	Alta	5
US14	Atualizar status	Como administrador, quero alterar o andamento do pedido, para informar a operação. Status limitado a recebido, preparo, entrega, concluído ou cancelado.	Alta	3
US15	Usar em diferentes telas	Como usuário, quero navegar pelo celular ou computador, para comprar com conforto. Layout responsivo sem rolagem horizontal em 390 px e funcional em desktop.	Média	5

3. Requisitos e regras de negócio

RF01–RF15 cobrem contas, catálogo, carrinho, checkout, pedidos, administração e tratamento de erros. RNF: responsividade, foco visível, variáveis de ambiente, PostgreSQL em produção, isolamento, reprodutibilidade e CI.

Regra	Descrição
RN01	Quantidade por item entre 1 e 20.
RN02	Somente produtos ativos podem ser comprados.
RN03	Preços e totais são recalculados no servidor.
RN04	Itens preservam nome e preço da compra.
RN05	Checkout cria pedido e itens atomicamente.
RN06	Pix simulado, cartão na entrega ou dinheiro.
RN07	Status limitado ao fluxo operacional definido.
RN08	Pedido só é visível ao proprietário/admin.

4. UML e fluxos

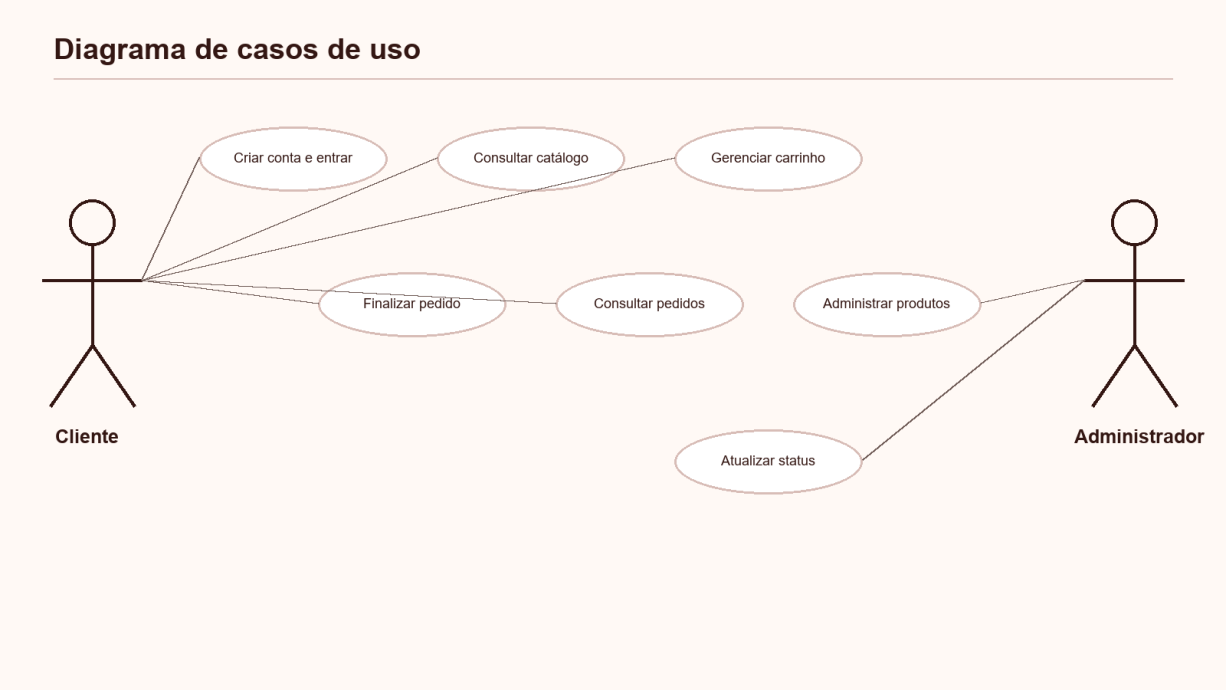


Figura 2 — Casos de uso.

Casos de uso expandidos

Caso	Ator	Pré-condição	Fluxo / resultado	Alternativas
Cadastrar cliente	Visitante	Não autenticado	Valida dados, cria usuário e inicia sessão.	Duplicidade/senha inválida mantém o formulário.
Finalizar pedido	Cliente	Carrinho válido	Revalida, cria pedido/itens e esvazia sessão.	Vazio, inativo ou inválido não cria pedido.
Atualizar status	Admin	Acesso ao Admin	Escolhe status permitido e salva.	Status inválido é rejeitado.

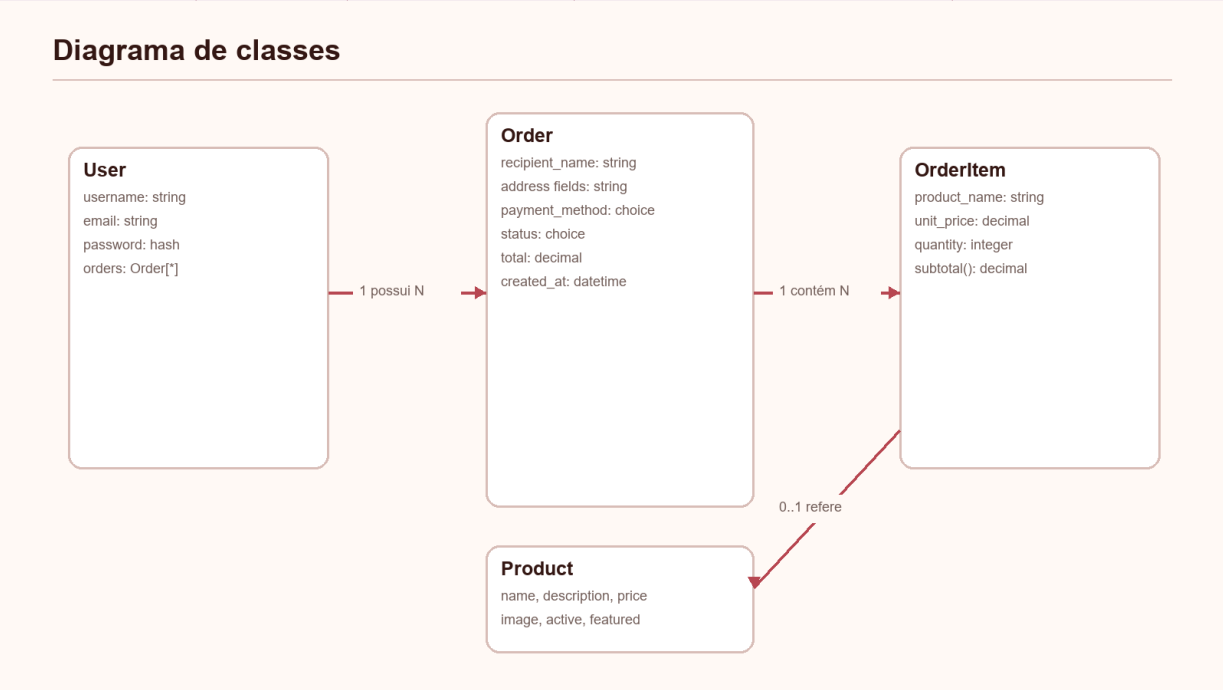


Figura 3 — Classes.

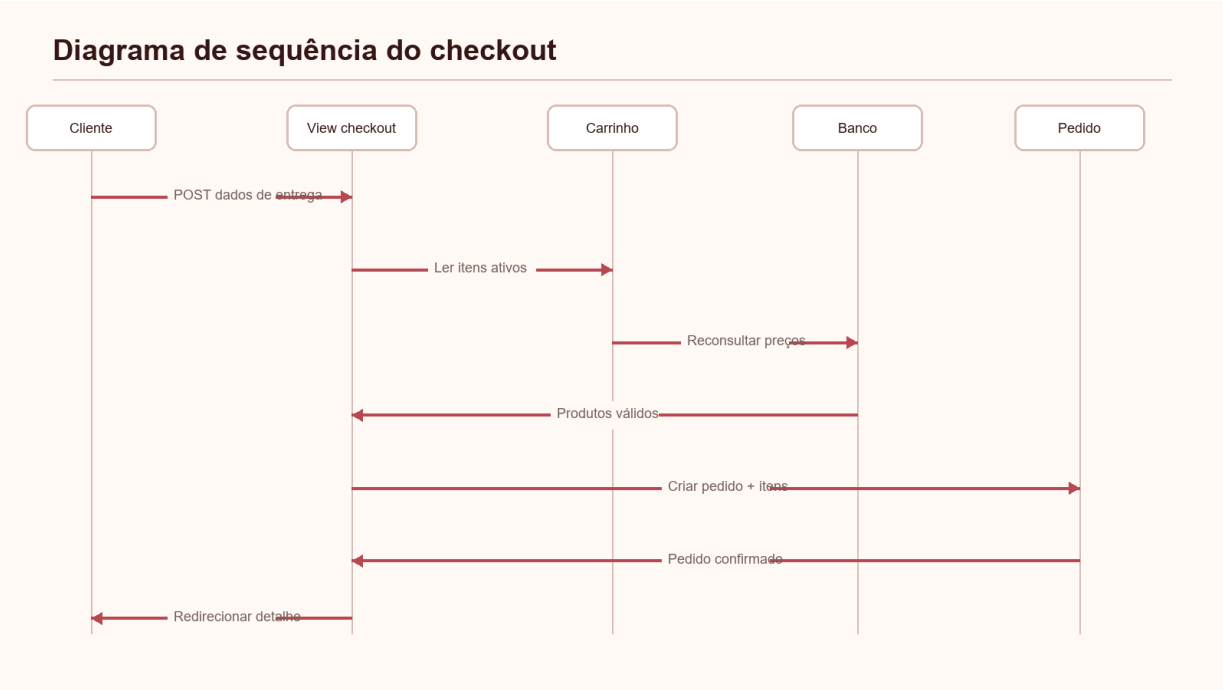


Figura 4 — Sequência do checkout.

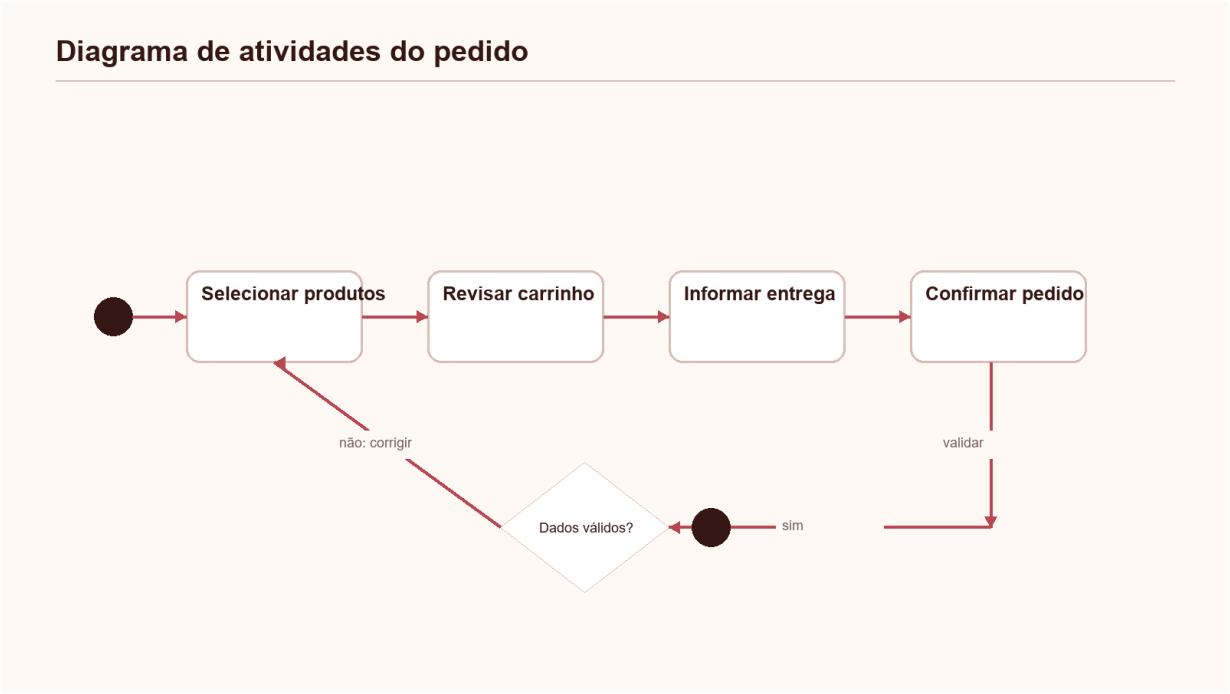


Figura 5 — Atividades.

5. Arquitetura e interface

Monólito com templates renderizados no servidor: navegador → views/formulários/serviço de carrinho → ORM → banco. Gunicorn serve a aplicação e WhiteNoise os estáticos. A marca combina rosa, creme e chocolate e usa imagens locais demonstrativas.

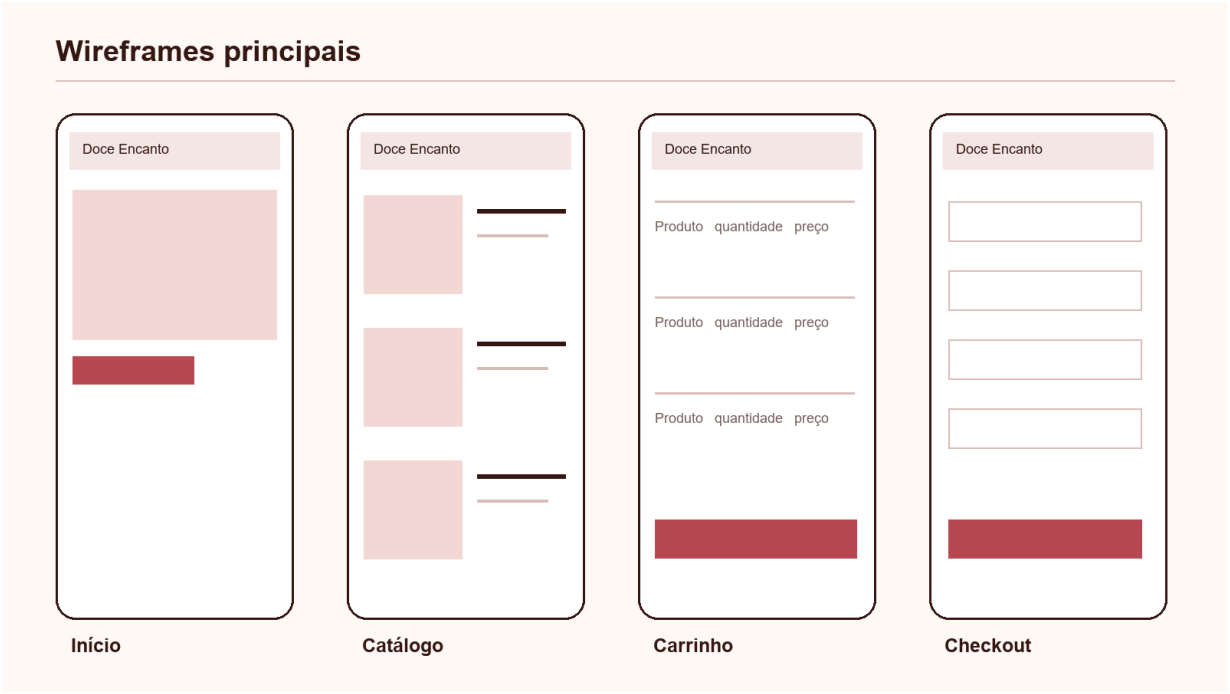


Figura 6 — Wireframes.

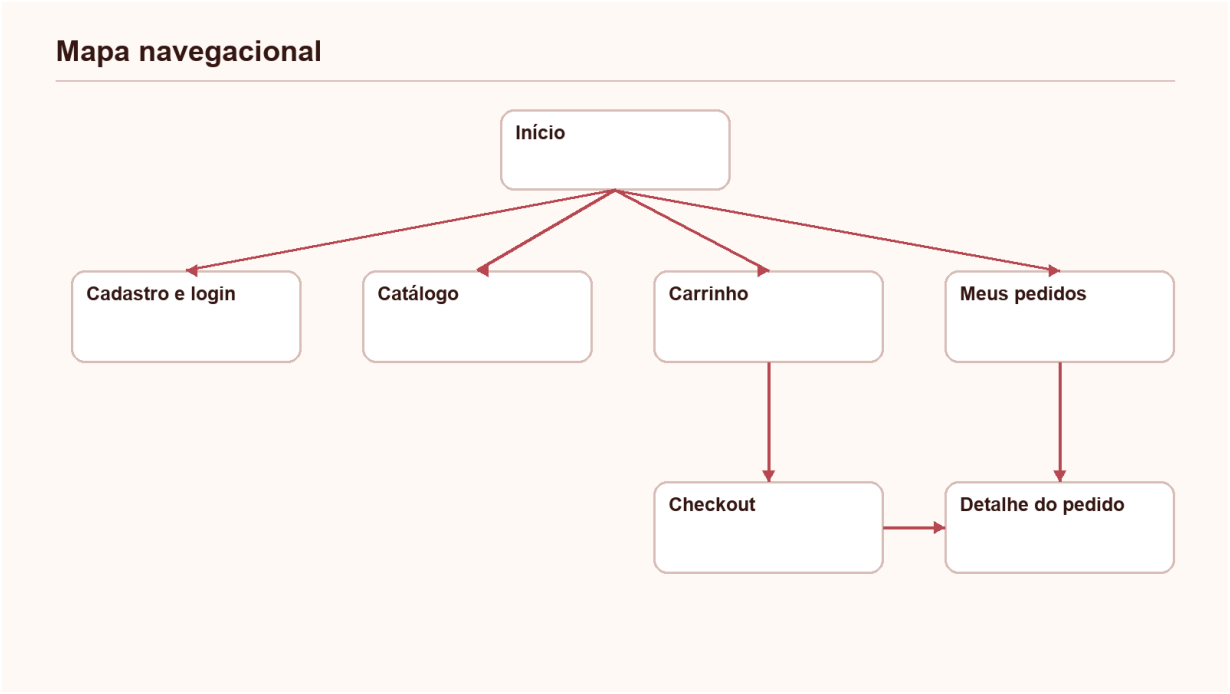


Figura 7 — Navegação.

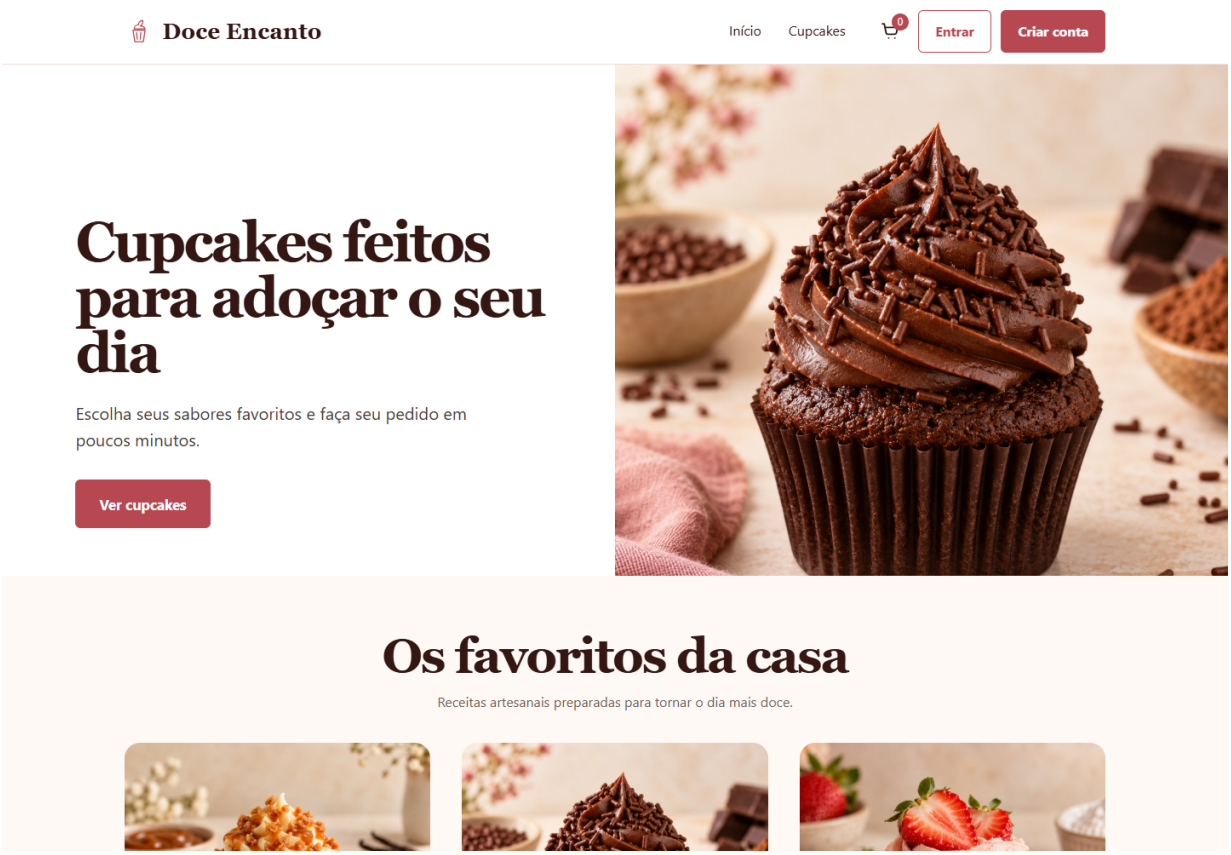


Figura 8 — Homepage desktop 1440×1000.

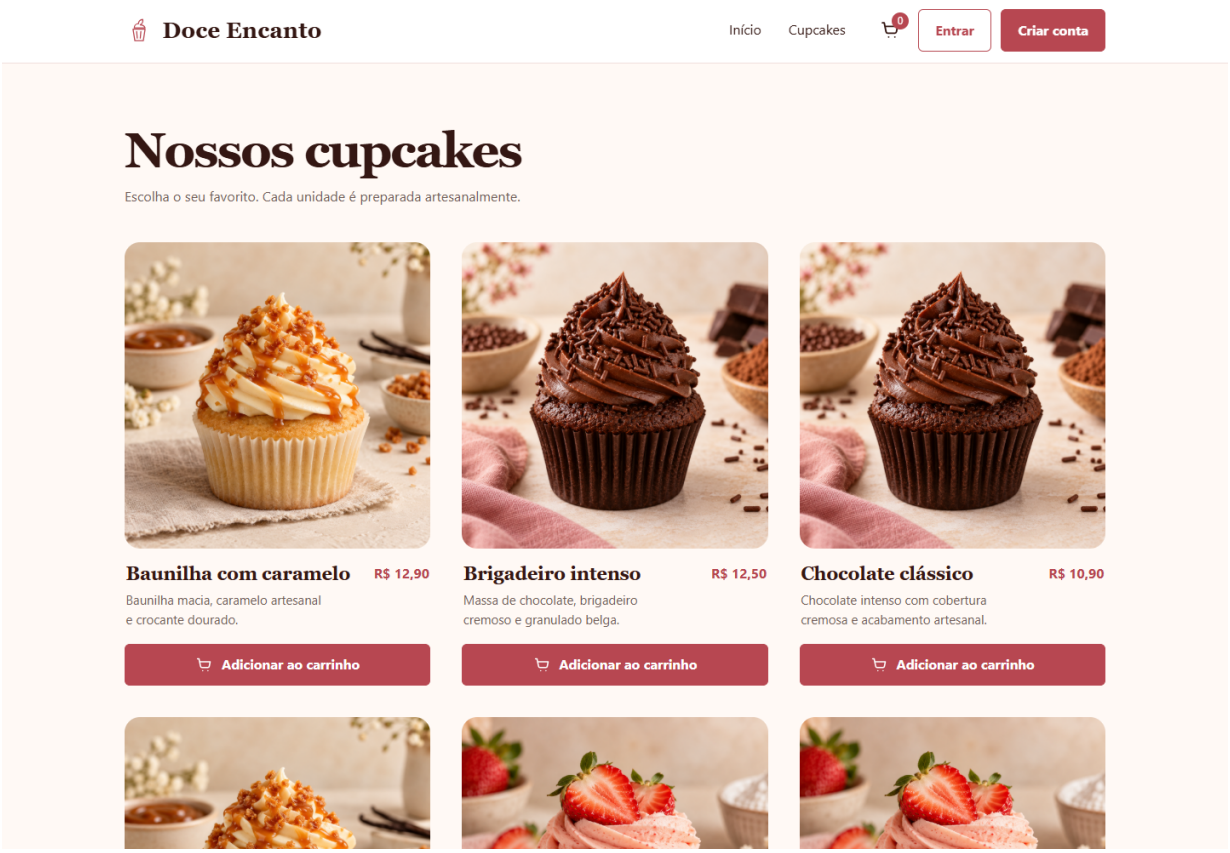


Figura 9 — Catálogo desktop.



Figura 10 — Homepage mobile 390×844.

6. Banco de dados

Um usuário possui muitos pedidos; cada pedido contém itens; o item referencia opcionalmente o catálogo e guarda snapshots. Essa separação mantém histórico mesmo se o produto mudar ou for removido.



Figura 11 — Modelo lógico.

Entidade	Campos centrais	Integridade
Product	name, description, price, image, is_active, timestamps	price ≥ 0
Order	user, address, payment_method, status, total, timestamps	FK usuário; escolhas controladas
OrderItem	order, product?, product_name, unit_price, quantity, subtotal	1..20; snapshot histórico
User	username, email, password hash	Autenticação nativa Django

7. Testes e laudo de qualidade

Resultado final: 16 testes automatizados aprovados, zero falhas e 87% de cobertura. O Django check não apontou problemas. O workflow do GitHub executa a suíte em push e pull request.

Sessão	Funcionou	Achado/correção	Fora do escopo
QA-01 — sessão interna simulada	Cadastro, login, catálogo e saída funcionaram de ponta a ponta.	Nenhuma falha impeditiva. A ordem dos destaques não seguia a curadoria visual; corrigida para preservar a ordem de cadastro.	Pagamento real, fora do escopo.
QA-02 — sessão interna simulada	Carrinho incluiu, atualizou e removeu itens; total permaneceu consistente.	Limite acima de 20 foi bloqueado/corrigido no servidor e coberto por teste.	Cupons, fora do escopo.
QA-03 — sessão interna simulada	Checkout criou pedido, esvaziou o carrinho e preservou preço/descrição.	Formulário inválido mantém os dados e explica os campos; sem defeito remanescente.	Gateway de pagamento, explicitamente simulado.
QA-04 — sessão interna simulada	Histórico e detalhe exibiram somente pedidos do usuário conectado.	Tentativa de abrir pedido de outro usuário retorna 404, validada por teste automatizado.	Rastreamento por mapa, fora do escopo.
QA-05 — sessão interna simulada	Homepage e catálogo aprovados em 1440×1000 e 390×844.	Foi encontrado erro 404 de favicon no smoke inicial; criado favicon local e console ficou sem erros.	Aplicativo nativo, fora do escopo.

Laudo

Defeito	Correção	Verificação
Ordem dos destaques	Ordenação por criação.	Teste específico aprovado.
Favicon 404	SVG local.	Console: 0 erros/avisos.

Defeito	Correção	Verificação
Total manipulável	Recálculo no servidor.	Testes de total/checkout.
Pedido alheio	Filtro por id + usuário.	Retorno 404 testado.
Pedido parcial	transaction.atomic.	Integração aprovada.

8. Manual e publicação

Local: clone o repositório; crie e ative .venv; instale requirements.txt; execute migrate, seed_demo e runserver. Teste com python manage.py test. Cliente: selecione itens, ajuste o carrinho, entre, finalize com pagamento simulado e acompanhe em Meus pedidos. Admin: acesse /admin/, gerencie produtos e atualize status.

Repositório público: https://github.com/Mathauz/doce-encanto. Publicação: conecte o repositório no Render e crie o Blueprint definido em render.yaml. A URL será inserida após a autenticação do aluno no Render.

9. Roteiros

Vídeo funcional — cerca de 5 minutos

Tempo	Ação
0:00–0:30	Problema, marca e pagamento acadêmico.
0:30–1:15	Homepage e catálogo.
1:15–2:00	Cadastro/login.
2:00–2:45	Carrinho e total.
2:45–3:40	Checkout.
3:40–4:20	Histórico e isolamento.
4:20–4:50	Admin e status.
4:50–5:00	GitHub, testes e URL.

Vídeo de correções — até 5 minutos

Tempo	Ação
0:00–0:35	Explicar QA simulada.
0:35–1:25	Ordem dos destaques.
1:25–2:05	Favicon e console.
2:05–2:55	Limites e totais.
2:55–3:40	Transação e snapshots.
3:40–4:20	Isolamento 404.
4:20–5:00	16 testes, 87% e CI.

10. Conclusão e referências

A solução atende ao fluxo planejado e cobre os principais riscos com validação no servidor, transações, permissões, testes e documentação. Referências: documentação Django 5.2; Render Deploy Django e Free Instances; materiais institucionais PIT I e PIT II.